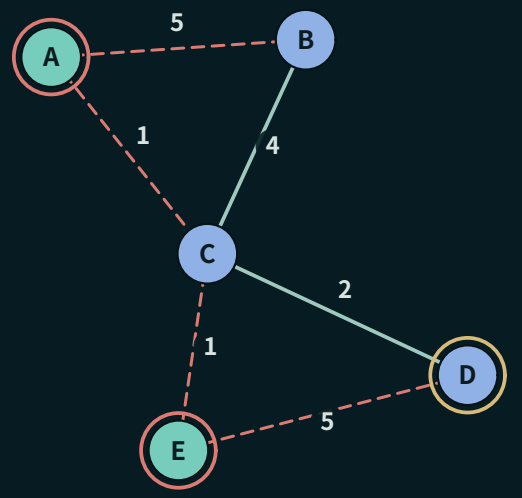
# 合法的那一步

Fiduccia–Mattheyses refinement · 单点移动与平衡窗口

Learning Clustering · 原作者 stephQ-lab（Issue #7 附件）

# 问题定义与 KL 的差别

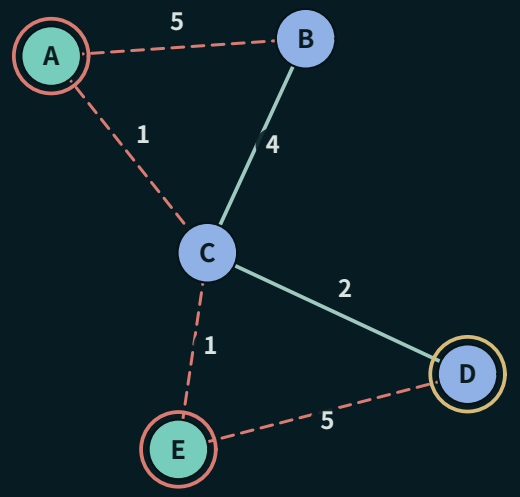
- 输入：图（或多引脚网表）、初始二分、maxDiff
- 操作：一次移动一个节点
- 合法：移动后两侧非空且 $|n_0 - n_1| \leqslant \text{maxDiff}$
- 输出：试探移动、累计收益、最佳前缀、接受的二分
- 复习：收益、锁定、试探、最佳前缀与 Module 07 相同




| 节点 | 侧 | 收益 | 合法 |
|---|---|---|---|
| A | 0 | +6 | 否 |
| B | 1 | +1 | 是 |
| C | 1 | -4 | 是 |
| D | 1 | +3 | 是 |
| E | 0 | +6 | 否 |

# 最大的那个不一定能动

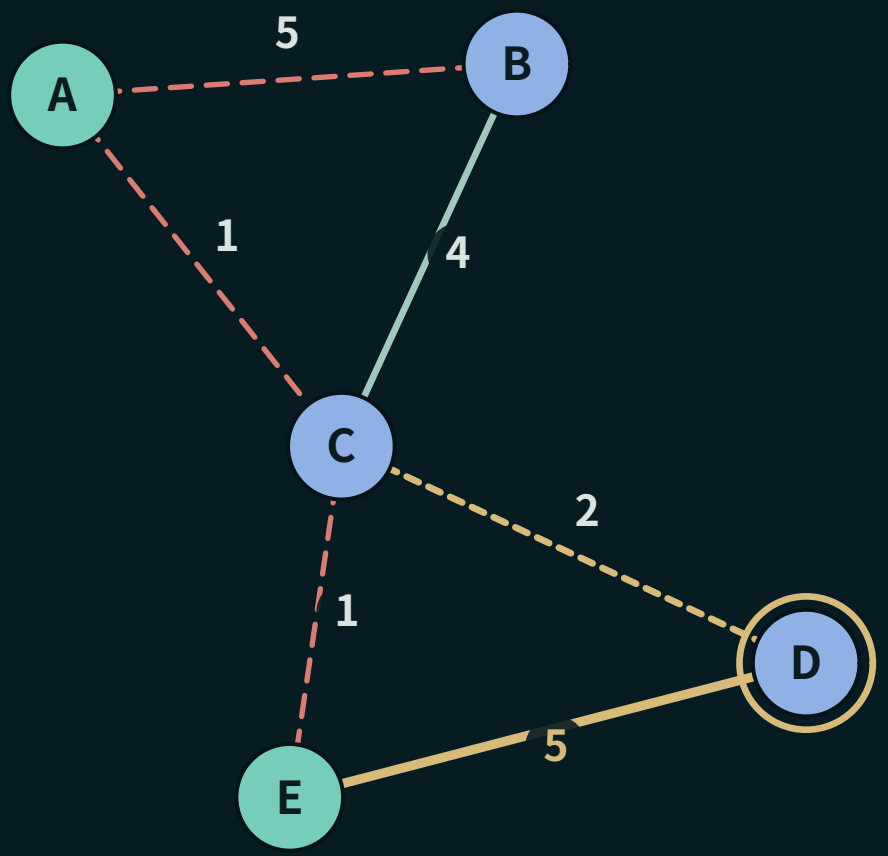
- 种子 AE | BCD，maxDiff = 1：只能 2:3 或 3:2
- A、E 收益 +6，但在只有 2 人的左侧
- 移走后 1 : 4：不合法
- 合法候选 B +1、C −4、D +3 → 选 D




| 节点 | 侧 | 收益 | 合法 |
|---|---|---|---|
| A | 0 | +6 | 否 |
| B | 1 | +1 | 是 |
| C | 1 | -4 | 是 |
| D | 1 | +3 | 是 |
| E | 0 | +6 | 否 |

# 收益的分解：FS 与 TE

- FS：v 是某 net 在本侧唯一的引脚 → +w
- TE：某 net 在目标侧没有引脚 → −w
- D：FS = D–E（5），TE = C–D（2），收益 3
- 二端边上：FS − TE = 外部 − 内部



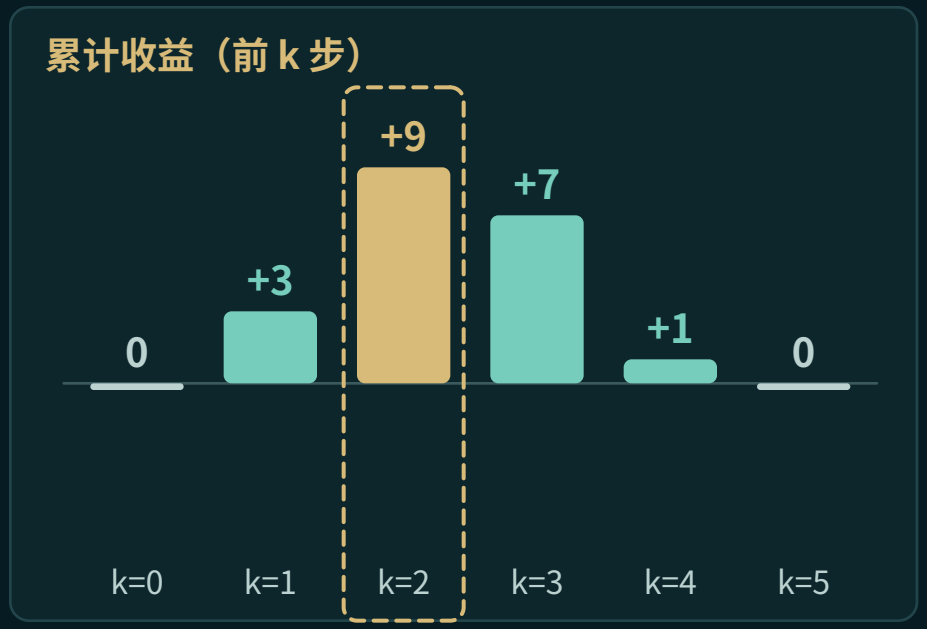
# 走完一轮

- D +3（cut 9）→ 左侧 3 人，A 变合法
- A +6（cut 3）
- 之后 C −2、E −6、B −1：照样试探
- cut 轨迹 12, 9, 3, 5, 11, 12

| 步 | 移动 | 收益 | cut | 累计 |
|---|---|---|---|---|
| 1 | D | +3 | 9 | +3 |
| 2 | A | +6 | 3 | +9 |
| 3 | C | -2 | 5 | +7 |
| 4 | E | -6 | 11 | +1 |
| 5 | B | -1 | 12 | 0 |

# 最佳前缀与回退

- 累计收益 3, 9, 7, 1, 0
- k* = 2：只应用 D、A
- 接受 ABC | DE，cut 3
- 试探终点 cut 12 被丢弃；第二轮空前缀，停止




接受前 2 步：ABC | DE，cut 3

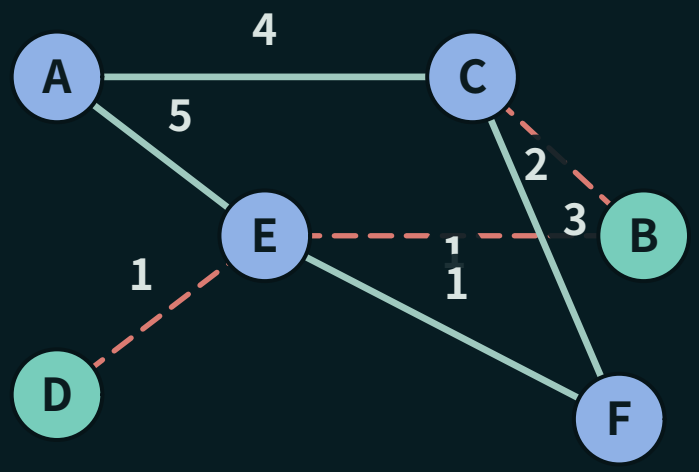
# 审查原附件的 hillclimb 说明

- 原文：初始 cut 9，合法收益全 ⩽ 0，最终 4
- 按附件数据：初始 cut 18
- E 合法且收益 +10
- 首轮最佳 8 = 穷举平衡最优 8
- 文字与数据不符：只作回归测试，不作教材

| | 原文 | 按数据计算 |
|---|---|---|
| 初始 cut | 9 | 18 |
| 合法收益 | 全部 ⩽ 0 | E 合法 +10 |
| 最终 cut | 4 | 首轮 8 |
| 穷举最优 | — | 8 |

# 替换反例：先负后正

- 6 节点，种子 ADE | BCF，maxDiff = 2，cut 6
- 初始所有合法收益 ⩽ 0：爬山法停在 6
- FM：A −1、B −1（cut 8），然后 E +4（cut 4）
- 保留 3 步：cut 4 = 穷举平衡最优
- 由固定种子搜索得到，测试中穷举核验

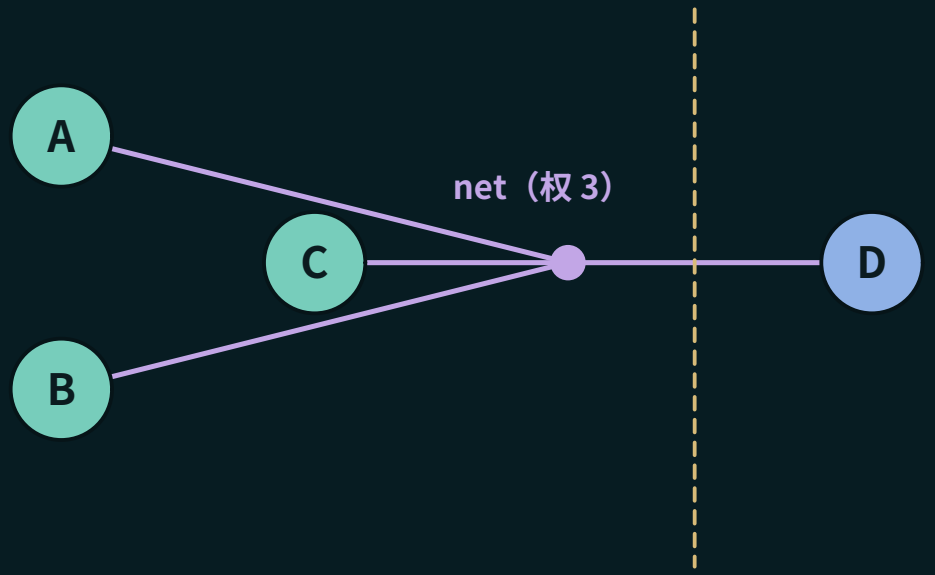



| 步 | 移动 | 收益 | cut |
|---|---|---|---|
| 1 | A | -1 | 7 |
| 2 | B | -1 | 8 |
| 3 | E | +4 | 4 |

保留 3 步：cut 6 → 4 = 穷举最优

# 桶、复杂度与多引脚 net

- 论文：桶链表 + 最大收益指针 → 每轮 O(P)
- 本课程：收益增量更新，但选择仍扫描——不是 O(P)
- 多引脚 net：按两侧引脚计数判定 FS/TE
- net{A,B,C,D}，D 在右：A 收益 0，D +3；拆成团会得 −1
- 原附件“hypergraph”案例只有二端边




按引脚计数：A 0，D +3；拆成团：A −1

# 伪代码逐行

- 第 2 行：按 net 计数算收益（gain）
- 第 3–4 行：只在合法候选中取最大（legal_moves）
- 第 5–6 行：试探、锁定、增量更新
- 第 7–8 行：最佳前缀与回退；第 9 行：空前缀停止

```
1  repeat：全部解锁，记下起点
2    按 net 计数，算出所有收益
3    while 有合法的未锁定节点:
4      v ← 合法候选中收益最大者
5      移动 v（试探），锁定 v
6      只更新受影响节点的收益
7    k* ← 最佳前缀（允许 0）
8    回到起点，只应用前 k* 步
9  until k* = 0
```

# 常见错误

- 直接选收益最大的节点（不看平衡）
- 照搬原附件的 hillclimb 说明
- 把“画了桶”当成“线性时间实现”
- 把二端边的例子叫作超图
- 以为人数平衡等于面积平衡

# 小结

## 先筛合法，再选最大；负收益也试探，接受只看最佳前缀。

反例要用独立计算核验；展示的桶不等于线性时间实现。

下一模块：目标从 cut 换成延迟，并允许复制节点。